DOCUMENT ID: ALBON-SE-2026-PBR
CLASSIFICATION: CONFIDENTIAL / INTERNAL USE ONLY
VERSION: 1.0 (PRODUCTION)

ALBON

DEEP TECH ENGINEERING

SOFTWARE ENGINEERING DESIGN BRIEF

Remote Monitoring and Control Challenge

A Modular IoT Stack for a Solar-Powered Photobioreactor Prototype

Issued By:	ALBON Software Engineering Core Team
Target Audience:	Software Engineering Candidates
Release Cycle:	Fiscal Year 2025/2026
Date of Issue:	May 25, 2026

Abstract

This design brief outlines a live software engineering challenge currently being addressed by the Software team at ALBON. The objective is to design and build a minimal, end-to-end remote monitoring and control stack for ALBON's solar-powered photobioreactor (PBR) prototype at Wiley Park. The system must accept operator input from a desktop client, publish data to a local web server through a clean API, and stream live sensor status to a web UI with safe hardware control actions. Candidates are expected to deliver a running proof-of-concept alongside architectural justification, with every major design decision traceable to reliability, safety, or maintainability reasoning. Use of AI development tools is encouraged and must be disclosed.

Contents

1. Software Engineering Mission

The Software Engineering team at ALBON builds reliable, remotely operated control and monitoring capability for early-stage hardware deployed in live wastewater environments by focusing on the following core objectives:

- Convert operational needs into robust software services and user interfaces.
- Deliver secure, observable APIs between field devices and cloud services.
- Collaborate with mechanical, electrical, and controls teams to translate hardware behaviour into software.
- Ensure stability, safety, and maintainability in real wastewater environments.

This challenge reflects a live problem the team is working through right now.

2. Context & Boundary Conditions

ALBON is iterating a solar-powered photobioreactor (PBR) prototype at Wiley Park to remediate wastewater using caged Intermediate Bulk Containers (IBCs) with pumps, sensors, and LED light rods. The unit operates unattended outdoors and requires continuous, precise monitoring of environmental conditions such as pH, dissolved oxygen, temperature, and turbidity to keep the algae alive and productive.

2.1 Existing Software Architecture

The current platform has no supervisory software layer. The Siemens LOGO! PLC executes deterministic, reactive safety logic at the hardware level, but there is no web-accessible interface, no remote monitoring capability, and no structured data logging. Operators must be physically present on-site to observe system state and issue commands.

2.2 Design Boundary

The software stack to be designed runs on top of the existing PLC hardware without modifying PLC ladder logic. The candidate system interfaces with the PLC through a software abstraction layer and must expose sensor data and actuator control to operators via a web UI accessible over a local area network. Cloud or wide-area network deployment is out of scope for this brief but must be considered in the hardening plan.

2.3 Operating Environment

The system runs on a developer laptop during development and targets a local-LAN deployment at the Wiley Park site for demonstration. Operators access the UI via a standard web browser. The physical environment is outdoors; assume intermittent network conditions and that a dropped connection must not leave actuators in an unsafe state.

3. Concept: Remote Monitoring and Control MVP

We require a lightweight, modular IoT software stack that reads sensors, issues commands to actuators, and presents live system status to operators via a web interface with safe intervention paths. The deliverable is a running proof-of-concept, not a slide deck.

3.1 Platform Infrastructure

The system spans three tiers: a desktop client that simulates or proxies sensor input and operator commands; a local web server that exposes a clean API and manages data flow between the client and the UI; and a browser-based web UI that displays live telemetry and provides basic hardware control. All tiers run on a single developer laptop for demonstration. The architecture must be modular enough to deploy the server tier to a Raspberry Pi or edge device without rewriting the UI or client layers.

3.2 The Architectural Optimisation Paradox

Candidates must balance several conflicting design directives:

- **Simplicity (Primary Driver):** this is a prototype for a small team. A clean, readable, two-file solution that works reliably beats a six-service architecture that requires a PhD to run. Every layer of complexity must earn its place.
- **Safety and Control Integrity:** the UI connects to real physical hardware. A stale command, a dropped connection, or an unvalidated input can actuate a pump or valve. The architecture must make unsafe states structurally impossible, not just unlikely.
- **Live Update Latency:** operators need to see what the system is doing right now, not ten seconds ago. The choice between polling, WebSockets, and server-sent events directly affects operator safety and confidence. The trade-off must be explicit and defended.
- **Portability:** the system must run on a developer laptop today and on a Raspberry Pi at the Wiley Park site next month. Technology choices that tie the stack to a specific OS, runtime version, or cloud provider create real operational risk.

3.3 Approach Selection Guidance

Candidates must propose and evaluate at least two architecture approaches (for example, REST with polling versus REST with WebSocket push, or an MQTT bridge versus a direct HTTP API) before selecting one. The selection must be justified against the directives above. There is no single correct answer, but the reasoning must be explicit and traceable to the operating constraints.

4. Design Criteria & Architectural Directives

The proposed system must satisfy the following design criteria:

4.1 Reliability and Performance

- Live update latency must be acceptable under local-LAN constraints. Define what ‘acceptable’ means for this use case and show the chosen architecture meets it.
- Graceful degradation on network loss: the system must detect a dropped connection and display a clear status to the operator rather than silently showing stale data.
- Safe recovery on reconnect: actuator state must be re-confirmed before any control commands are re-enabled after a connection drop.

4.2 Safety and Control Integrity

- Clear command paths with explicit confirmation and acknowledgement for every actuator write action.

- Basic role separation between view-only and control access, documented even if not fully implemented in the prototype.
- Guardrails for unsafe commands: out-of-range setpoints must be rejected before reaching the hardware layer.
- Dry-run mode for demonstration: the system must be demonstrable without physical PLC hardware attached.

4.3 Modularity and Maintainability

- Separation of concerns across UI, API, data model, and device or service adapter layers.
- Object-oriented or component patterns with clear interfaces so that the simulated device adapter can be swapped for a real PLC adapter without touching the UI or API.
- Tests for at least one critical component, covering both the happy path and at least one failure mode.

4.4 Observability

- Structured logs on the server with timestamps, log levels, and consistent field names.
- Minimal metrics: at minimum, message rate, error rate, and round-trip latency between client and server.
- A brief troubleshooting checklist covering the three most likely failure modes in a live site deployment.

4.5 Security

- Local-only default bind on all server sockets: the system must not be accidentally exposed to the internet during development.
- Input validation on all API endpoints: types, ranges, and allowed values must be checked server-side before any command is processed.
- Basic authentication strategy documented, even if not fully implemented: how would you lock this down for WAN access?

4.6 Scalability and Portability

- The device abstraction layer must allow sensors and actuators to be swapped or added without changes to the UI or API contract.
- The system must run on a developer laptop with a single setup command.
- A clear path to containerise and deploy the server tier to a Raspberry Pi or equivalent edge hardware must be documented.

5. Challenge Brief

Propose and build a remote monitoring and control MVP that meets the criteria above. Present the concept, defend key design choices and trade-offs, and demonstrate a working slice of the system.

5.1 Your submission must include:

5.1.1 Ideation Phase

- Two architecture sketches covering meaningfully different approaches (for example: REST with polling versus REST with WebSocket push; MQTT bridge versus direct HTTP API; synchronous request-response versus event-driven).
- Pros and cons for each approach against the directives in Section 4.
- A clear selection decision with explicit reasoning tied to the operating constraints.

5.1.2 System Concept

- A block diagram (mandatory) showing the desktop client, API or web server, data flow, and web UI.
- An API definition covering endpoints, payloads, authentication approach, and error model.
- The chosen live update method (polling, WebSocket, server-sent events) with quantified latency expectations and rationale.
- Safety measures and fail-safe logic for all actuator write actions, including what happens on connection loss.

5.1.3 Engineering Considerations

- **Software:** module boundaries, classes or interfaces, threading or async model, and test coverage.
- **Data:** schema for sensor readings and command messages, and a retention plan for logged data.
- **Process:** development environment setup, run instructions, and the path to deploy the server tier to edge hardware.
- **Ops:** logging format, minimal metrics, and a troubleshooting checklist for the three most likely live-site failure modes.
- **Security:** threat assumptions for local-LAN use and a documented hardening plan for WAN or cloud deployment.
- **Cost:** a brief bill of materials covering services, libraries, and hosting, with an estimated monthly cost at pilot scale.

5.1.4 Demonstration Requirements

- A desktop application or script that accepts operator inputs (buttons, text fields, or CLI commands) and publishes them to the server.
- A local web server hosting a UI that displays the desktop inputs and live sensor data in real time.
- A working API between client and server, demonstrable without physical PLC hardware.
- Clear code comments and a short code-style guide in the repository README.
- Unit or integration tests for at least one critical component, covering a happy path and at least one failure mode.
- A README that allows ALBON engineers to run the full stack on a developer laptop in under 10 minutes.

6. Engineering Considerations

Candidates should explicitly address the following cross-cutting concerns in their submissions:

- **Concurrency:** how simultaneous operator commands and background sensor polling interact, and whether the chosen threading or async model can handle both safely.
- **State management:** how the server tracks the current state of each actuator, what happens if the server restarts mid-operation, and how clients are notified of state changes they did not initiate.
- **Error propagation:** how errors from the device layer (simulated PLC faults, sensor timeouts) surface to the operator UI in a way that is actionable rather than alarming.
- **Portability:** specific Python version, Node version, or runtime dependencies that would prevent the stack from running on a Raspberry Pi 4 running Raspberry Pi OS Bookworm.

7. Interview Discussion Framework

Be ready to defend the following specific technical perspectives during the second-interview panel. The interview will reference the Wiley Park PBR system as the operational context; this is a practical software engineering assessment, not a memorisation exercise.

7.1 Explanation of Design Choices

- Defend the chosen architecture against the alternative developed during ideation, with specific reference to the latency, safety, and portability constraints.
- Justify the live update method: what latency did you measure or estimate, and how does it translate to operator confidence in a real site deployment?
- Walk through the fail-safe behaviour end-to-end: from connection drop detection to actuator state confirmation on reconnect.

7.2 Tuning, Failure Modes, and Path to Production

- Identify the failure mode most likely to surface first in a live Wiley Park deployment and the diagnostic path a technician would follow to find it.
- Describe how the device abstraction layer would be replaced with a real python-snap7 or pyModbusTCP adapter, and what changes that would require in the rest of the stack.
- The path from this prototype to a small-batch site deployment: what changes, what stays, and what gets formalised before handing to a site operator.

8. Notes and Guidance

- State all assumptions and constraints explicitly. Quantify wherever possible.
- Prefer integrated solutions over one-off scripts. A clean two-tier prototype that works reliably beats a clever workaround.
- Candidates are not expected to be algae specialists. Ask questions and state uncertainties up front.
- Use metric units only. Latency in milliseconds, data rates in bytes per second or messages per second.

- Keep diagrams clear and label them properly. Cite key references or library documentation at the end of the submission.
- Using AI is encouraged. Disclose where, how, and what you verified afterwards. Hiding AI use counts against the candidate; disciplined use counts for them.

8.1 Submission Format

- **File:** PDF format, A4 layout, no page limit but concise is preferred (ideally <10 page body). An appendix should be used for supplementary figures; please include one or two high-level figures in the main body per design alternative.
- **Naming Convention:** ALBON_SE_[Surname]_[FirstName]_YYYYMMDD.pdf.
- **Email Address:** info@albon.com.au (Subject line: SE Brief – [Surname] [FirstName]).
- **Fonts:** 12 pt, any clean and readable sans-serif style font, clear headings.
- **Figures:** In-line, labelled. Vector or 300 dpi raster.
- **Repository:** Link to a public or private Git repository with build and run instructions. ALBON engineers must be able to run the demo in under 10 minutes.
- **AI Disclosure:** Note any AI or external tools used and how they assisted. Use of AI is encouraged at ALBON.
- **Accessibility:** Ensure colour-independent readability. Provide alt text for critical diagrams where possible.
- **Originality:** Your own work. Note any AI or external tools used.

8.2 Submission Terms

- **Confirmation:** By submitting, you confirm the work is your own and does not infringe third-party rights.
- **Ownership:** ALBON owns its pre-existing IP. You own your submission unless and until you join ALBON or we agree otherwise in writing.
- **Licence:** You grant ALBON a non-exclusive, royalty-free licence to review, test, and evaluate your submission for recruitment purposes.
- **Confidentiality:** Treat all non-public ALBON information in this brief and during interviews as strictly confidential. Do not share without written permission.
- **No Obligation:** ALBON is not obliged to use, develop, or purchase your submission.

References

[1] Raspberry Pi Ltd., Raspberry Pi 4 Model B Product Brief, Cambridge, UK, 2019. [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf

[2] Mozilla Developer Network, "WebSockets API," MDN Web Docs, Mozilla Foundation. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

[3] OASIS Standard, MQTT Version 5.0, OASIS, Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[4] Siemens AG, SIMATIC NET S7-CPs for Industrial Ethernet — Configuring and Commissioning, Manual Part A, Siemens Industry Online Support. [Online]. Available: https://support.industry.siemens.com/cs/ww/en/view/30374198